

CSE252 · INTRODUCTION TO ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

UNIT 2

Data Preparation and the Machine Learning Workflow

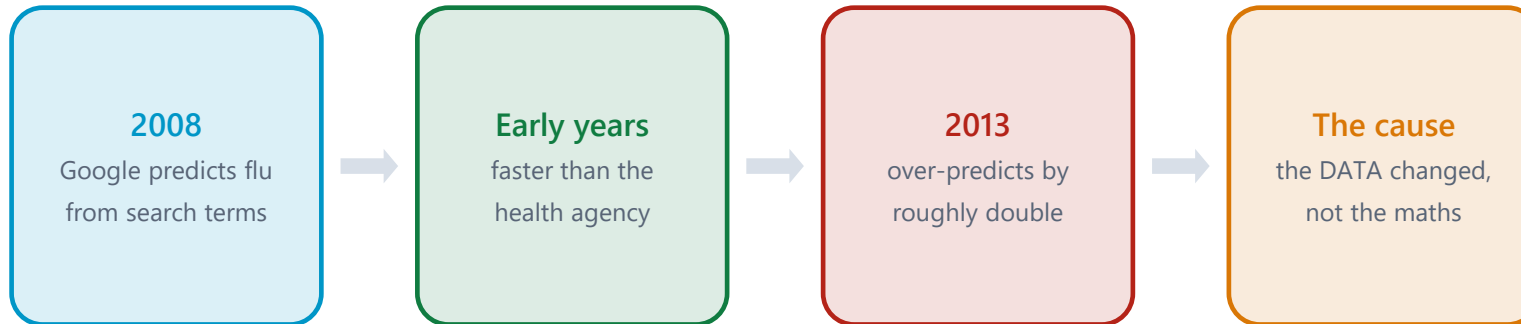
Models are only as good as the data beneath them. This unit is the 70-80% of every AI project that nobody puts in the demo video.

NumPy · Pandas · Matplotlib · Understanding Data & EDA · Missing Values & Outliers · Feature Engineering · Train-Test Split & Cross Validation

OPENING STORY

The Flu Tracker That Beat the Government - Then Broke

UNIT 2



HOW THE STORY GOES

2008 Google Flu Trends predicted flu outbreaks from what people searched for - and reported them faster than the official surveillance system.

It worked and was widely celebrated as a demonstration of what big data could do.

2013 it over-estimated peak flu by roughly double, and was eventually retired.

What broke not the model. Search behaviour changed, autocomplete changed, media coverage drove searches by people who were not ill.

The lesson the model was fine. The relationship between the data and the world had moved.

DID YOU KNOW?

This is CONCEPT DRIFT - the relationship between inputs and outcome changing over time. It is the single most common way a deployed model dies quietly.

THINK OF IT LIKE a thermometer someone repainted. the instrument still works perfectly; the scale no longer means what it did.

TAKEAWAY The algorithm never changed. The data did - and nobody was watching for it.

UNIT II ROADMAP

What This Unit Covers

UNIT 2

Everything here happens BEFORE you train a single model.

- Introduction to NumPy, Pandas and Matplotlib
 - The three libraries every ML workflow in Python is built on
- Understanding Data
 - Types of datasets, data collection, and Exploratory Data Analysis
- Data Preprocessing
 - Handling missing values; detecting and treating outliers
- Feature Engineering
 - Feature scaling, feature encoding, feature selection
- Training and Validation
 - Train-test split and K-fold cross validation
- Lab work attached to this unit
 - Data manipulation with NumPy and Pandas; visualisation with Matplotlib
 - EDA on the Titanic and Student Performance datasets
 - Preprocessing: missing values, encoding, feature scaling

WHAT YOU ALREADY BRING

What You Already Bring to This Unit

UNIT 2

Four bridges into this unit - including one from the course you are taking alongside this one.

FROM UNIT I

The AI lifecycle

- You named the six phases already
- Phases 2 and 3 - data and preparation - are this unit
- And you predicted phase 3 eats the most time

FROM PROGRAMMING

Lists, loops and files

- A DataFrame is a table you can index
- NumPy arrays are lists that do maths in bulk
- If you can write a loop, you can read this code

FROM STATISTICS

Mean, median, spread

- You know what an average is
- Median vs mean is the whole outlier story
- Standard deviation returns as the z-score

WHERE IT LANDS

Robot sensor data

- Lidar, IMU, encoders - the messiest data there is
- Different units, different rates, dropped packets
- Every technique today was built for exactly this

THE FOUR BRIDGES

1 Unit I gave you the lifecycle**2** programming gives you the tools**3** statistics gives you the checks**4** and sensors give you the mess

LEARNING OUTCOMES

Learning Outcomes

UNIT 2

By the end of this unit you should be able to:

1 Use NumPy and pandas to load, inspect and reshape a dataset. CO2

2 Choose the right plot for the question being asked. CO2

3 Diagnose missing data as MCAR, MAR or MNAR and justify an imputation strategy. CO2

4 Detect outliers with the IQR and z-score rules, and decide what to do with them. CO2

5 Build a leak-proof train / validation / test pipeline with scaling and encoding. CO2

Remember

recall the terms

Understand

explain them in your own words

Apply

use them on a problem you have not seen

Analyse

choose between them and justify it

the levels this unit is assessed at

01

SECTION

NumPy, Pandas and Matplotlib

IN THIS SECTION

- The Python scientific stack and why it exists
- NumPy - contiguous arrays and vectorisation
- Pandas - Series, DataFrames, and relational operations
- Matplotlib and Seaborn - seeing your data
- Why NumPy is ~100x faster than a Python loop

2.1 THE SCIENTIFIC STACK

The Python Scientific Computing Stack

UNIT 2

Four libraries, one pipeline: NumPy computes, Pandas organises, Matplotlib reveals, Scikit-learn learns.

NUMERICAL FOUNDATION

NumPy

- N-dimensional homogeneous array object (ndarray)
- Stored in one contiguous block of memory
- Vectorised operations compiled in C, using CPU SIMD
- Linear algebra, random sampling, broadcasting
- Everything else on this slide is built on top of it

TABULAR DATA

Pandas

- Series = 1-D labelled array; DataFrame = 2-D labelled table
- Heterogeneous columns - numbers, strings and dates together
- Indexing, filtering, groupby, merge/join, pivot
- Built-in missing-value handling (NaN)
- Reads CSV, Excel, SQL, JSON, Parquet

VISUALISATION

Matplotlib & Seaborn

- Matplotlib: full low-level control over every plot element
- Seaborn: statistical plots in one line, built on Matplotlib
- Histograms, box plots, KDE curves, scatter matrices
- Correlation heatmaps and pair plots
- Publication-quality output

WHERE THIS LEADS

Scikit-learn

- Consumes NumPy arrays and Pandas DataFrames
- Uniform API: fit(), transform(), predict()
- Every preprocessing tool in this unit lives here
- Every model in Units 3-5 lives here too

2.1 THE SCIENTIFIC STACK

UNIT 2

Why Is NumPy ~100x Faster Than a Python Loop?

EXAM TIP · The single most examined 'why' question about NumPy

Reason 1 - Contiguous memory a NumPy array is one unbroken block of memory. A Python list is a block of POINTERS to objects scattered across memory, so the CPU cache constantly misses.

Reason 2 - No per-element type checking every element in an ndarray has the same declared dtype. Python must check the type of every list element on every operation.

Reason 3 - Compiled C loops the loop runs inside compiled C code, not in the Python interpreter. No bytecode dispatch per element.

Reason 4 - SIMD vectorisation Single Instruction, Multiple Data - the CPU applies one instruction to several values simultaneously.

The practical rule if you are writing a `for` loop over a NumPy array, there is almost always a vectorised expression that is shorter, clearer and far faster.

PREDICT Adding two arrays of 1 million numbers: how long with a Python loop, and how long with NumPy? Write down both guesses.

2.1 VISUALISATION

Choosing the Right Plot

UNIT 2

The plot you choose is determined by how many variables you have, and their types.

ONE NUMERIC VARIABLE**Histogram / KDE**

- Shows the SHAPE of a distribution
- Reveals skew, multiple peaks, gaps
- `plt.hist()` / `sns.histplot(kde=True)`

ONE NUMERIC, BY GROUP**Box plot / Violin**

- Median, quartiles and outliers at a glance
- Best tool for comparing groups side by side
- `sns.boxplot(x='Pclass', y='Age', data=df)`

TWO NUMERIC VARIABLES**Scatter plot**

- Shows relationship, curvature and clusters
- Reveals what a correlation number hides
- `plt.scatter()` / `sns.scatterplot(hue='target')`

ONE CATEGORICAL**Bar chart / Count plot**

- Frequency of each category
- Immediately exposes class imbalance
- `sns.countplot(x='Survived', data=df)`

MANY NUMERIC**Correlation heatmap**

- All pairwise correlations in one image
- Spots redundant features fast
- `sns.heatmap(df.corr(), annot=True)`

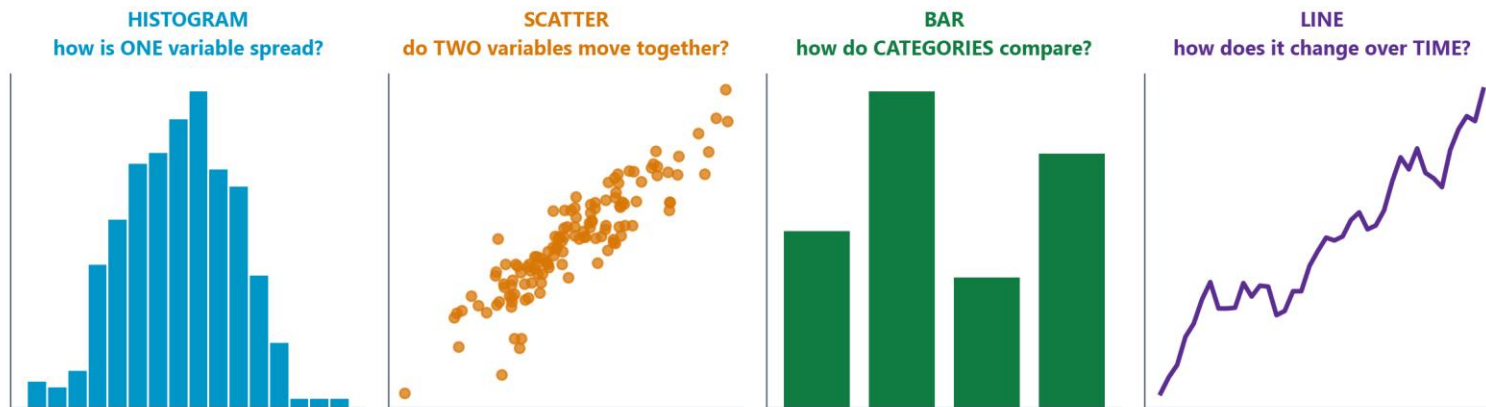
EVERYTHING AT ONCE**Pair plot**

- Scatter matrix of every numeric pair
- Colour by target to see class separation
- `sns.pairplot(df, hue='Survived')`

2.1 VISUALISATION

Four Plots, Four Different Questions

UNIT 2



HOW TO READ THIS FIGURE

Histogram how is ONE variable spread?
Shape, centre, skew, outliers.

Scatter do TWO variables move together?

Bar how do CATEGORIES compare?

Line how does something change over TIME?

The rule choose the plot from the QUESTION, never from what looks impressive.

TAKEAWAY Pick the chart that answers your question. A pie chart with eleven slices answers none.

CHECK YOUR UNDERSTANDING

Quick Check - Section 1

UNIT 2

QUICK CHECK

Why is a NumPy vectorised operation dramatically faster than the equivalent Python for-loop over a list?

- A** NumPy runs on the GPU instead of the CPU
- B** NumPy uses contiguous memory, fixed dtypes and compiled C loops with SIMD
- C** NumPy automatically parallelises across all CPU cores
- D** Python lists are stored on disk while NumPy arrays are in RAM

ANSWER: B The gains come from memory layout and execution: one contiguous block (cache-friendly), a single declared dtype (no per-element type checks), and a compiled C loop using SIMD instructions - instead of interpreted Python bytecode per element.

02

SECTION

Understanding Data and Exploratory Data Analysis

IN THIS SECTION

- Types of datasets - tabular, time-series, spatial, unstructured
- Data collection and what can go wrong at the source
- Univariate, bivariate and multivariate analysis
- Reading a correlation coefficient correctly
- EDA on Titanic and Student Performance

2.2 UNDERSTANDING DATA

Types of Datasets

UNIT 2

The data type dictates the tools, the split strategy, and the failure modes.

TYPE 1

Tabular / Structured

- Rows are instances, columns are features
- CSV files, SQL tables, spreadsheets
- Example: Titanic passengers, student records
- Most business data; the focus of Units 2-5

TYPE 2

Time-Series

- Observations ordered in time at regular intervals
- IoT sensor telemetry, stock prices, energy demand
- ORDER MATTERS - never shuffle before splitting
- Requires time-aware validation, not random splits

TYPE 3

Spatial / Geographic

- Coordinates, GIS shapefiles, GPS trajectories
- Delivery routing, crop monitoring, urban planning
- Nearby points are correlated - spatial autocorrelation
- Distance is a meaningful feature

TYPE 4

Unstructured

- Text corpora, images, audio waveforms, video
- No fixed row-column structure
- Must be converted to numeric vectors before modelling
- Where deep learning dominates - Unit 6

ASK THE CLASS Why can you not randomly shuffle a time-series before doing a train-test split?

2.2 DATA COLLECTION

Data Collection - Where Your Data Comes From, and What Goes Wrong

UNIT 2

No preprocessing technique can repair data that was collected badly. The cheapest fix is always at the source.

Common sources

Primary collection surveys, experiments, sensors and instrumentation you control. Highest quality, highest cost.

Operational systems transaction logs, CRM records, application databases. Abundant, but collected for a different purpose.

Public / open data government portals, Kaggle, UCI ML Repository, WHO and World Bank datasets.

Web scraping and APIs large volumes quickly - but check the terms of service and the licence before you use anything.

Third-party purchase commercial data vendors. Verify provenance and consent, not just the schema.

What goes wrong at the source

Sampling bias the sample does not represent the population. An online survey about internet access cannot reach people without internet access.

Survivorship bias you only see the records that survived a filter. Analysing only current customers hides everything about the ones who left.

Measurement error miscalibrated sensors, inconsistent units, data-entry mistakes, duplicated records.

Label noise the ground truth itself is wrong - a mislabelled scan, a misclassified transaction.

Consent and legality personal data carries obligations. 'Publicly available' is not the same as 'free to use'.

ASK THE CLASS A university surveys students about campus WiFi quality using an online form. Which bias is baked in before a single response arrives?

2.2 EXPLORATORY DATA ANALYSIS

UNIT 2

Exploratory Data Analysis - Three Levels

LEVEL 1

Univariate - one variable at a time

- Statistics: mean, median, mode, std deviation, skewness
- Plots: histogram, box plot, KDE density curve
- For categories: value counts and bar charts
- Questions: what is typical? how spread out? is it skewed? any impossible values?

LEVEL 2

Bivariate - two variables together

- Numeric vs numeric: scatter plot, Pearson correlation r
- Category vs numeric: box plot grouped by class
- Category vs category: cross-tabulation, stacked bar
- Questions: do they move together? does the target differ by group?

LEVEL 3

Multivariate - many at once

- Correlation heatmap across all numeric features
- Seaborn pair plot with hue set to the target
- Parallel coordinates plots
- Dimensionality reduction projections (PCA - Unit 4)
- Questions: which features are redundant? do clusters exist?

WHY BOTHER

What EDA is actually for

- Find errors before they poison the model
- Discover which features carry signal
- Detect missing values, outliers and imbalance
- Form hypotheses that guide feature engineering
- Catch leakage - a feature that is 'too good' is usually a bug

2.2 EXPLORATORY DATA ANALYSIS

UNIT 2

Reading a Correlation Coefficient - and Its Traps

PEARSON CORRELATION COEFFICIENT

$$r = \text{cov}(X, Y) / (\sigma_x \cdot \sigma_y) \quad r \in [-1, +1]$$

Measures the strength and direction of a LINEAR relationship only.

HOW TO READ R

$r = +1$ perfect positive linear relationship

$r \approx 0$ no LINEAR relationship - but there may be a strong curved one

$r = -1$ perfect negative linear relationship

$|r| > 0.7$ strong; 0.3-0.7 moderate; < 0.3 weak (rules of thumb only)

Trap 1 correlation is NOT causation

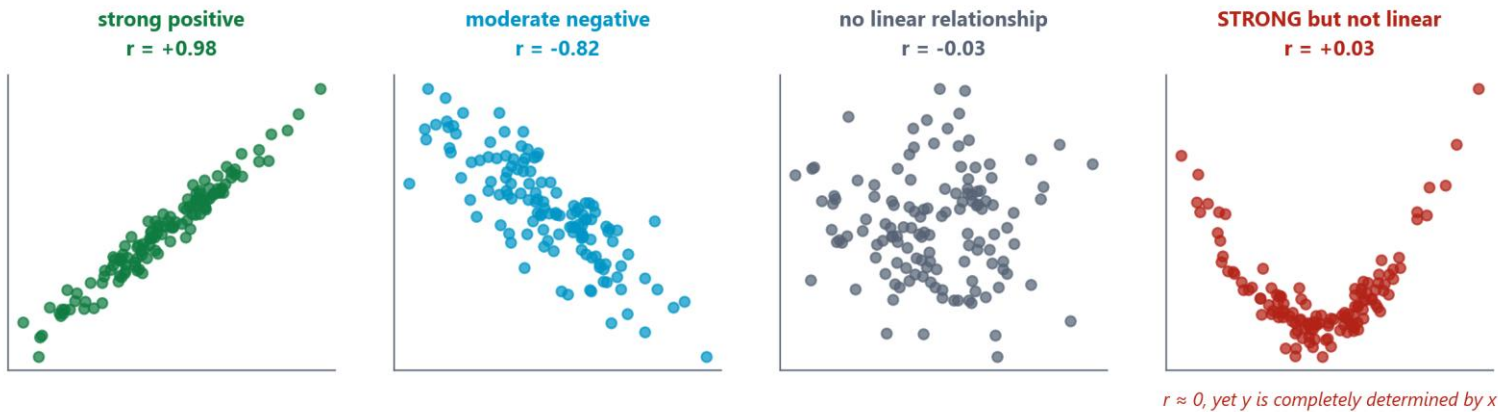
Trap 2 $r = 0$ does not mean independent - a perfect parabola has $r \approx 0$

Trap 3 r is highly sensitive to outliers - always plot the scatter too

2.2 EXPLORATORY DATA ANALYSIS

What the Correlation Coefficient Does Not Tell You

UNIT 2



HOW TO READ THIS FIGURE

Panels 1 and 2 r captures the strength and direction of a LINEAR relationship.

Panel 3 $r \approx 0$ and there really is no relationship. Fine.

Panel 4 $r \approx 0$ and y is COMPLETELY determined by x . The relationship is a curve.

The lesson r near zero does not mean 'no relationship'. It means 'no straight-line relationship'.

So always plot it. A correlation matrix alone will lie to you by omission.

TAKEAWAY $r = 0$ does not mean independent. Panel 4 is a perfect relationship with $r \approx 0.03$.

2.2 EXPLORATORY DATA ANALYSIS

EDA in Practice: Titanic and Student Performance

UNIT 2

REAL-WORLD APPLICATION · The two datasets from your lab practicals

Titanic - the survival question each row is a passenger; the target is Survived (0/1). Features include Sex, Pclass, Age, Fare, Embarked.

Bivariate finding 1 cross-tabulating Survived against Sex is stark - roughly 74% of women survived versus roughly 19% of men. 'Women and children first' is visible directly in the data.

Bivariate finding 2 survival by passenger class: around 63% in 1st class, falling sharply through 2nd and 3rd. Socio-economic status predicts survival.

Missing data reality the Age column is missing for roughly 20% of passengers, and Cabin is missing for most. This dataset is a preprocessing exercise, not just a modelling one.

Student Performance correlation matrices show a clear positive association between parental education level and the final grade G3, and between prior-period grades and G3.

The transferable lesson in both datasets, EDA told you which features mattered BEFORE any model was fitted. That is the point of the exercise.

DISCUSS The Titanic 'Cabin' column is missing for most passengers. Do you drop the column, or is the missingness itself informative?

CHECK YOUR UNDERSTANDING

Quick Check - Section 2

UNIT 2

QUICK CHECK

You compute the correlation between two features and get $r = 0.02$. What can you correctly conclude?

- A** The two features are independent of each other
- B** There is no linear relationship, but there may be a strong non-linear one
- C** One feature causes almost no change in the other
- D** The features are useless and both should be dropped

ANSWER: B Pearson's r measures LINEAR association only. A perfect parabola gives $r \approx 0$ despite a completely deterministic relationship. Always plot the scatter before drawing conclusions - and never read causation into a correlation.

03

SECTION

Data Preprocessing: Missing Values and Outliers

IN THIS SECTION

- The three mechanisms of missingness: MCAR, MAR, MNAR
- Imputation strategies and their trade-offs
- Z-score outlier detection for symmetric data
- The IQR rule for skewed data
- When an outlier is an error and when it is the signal

2.3 MISSING VALUES

UNIT 2

Three Mechanisms of Missing Data

Before you fill a gap, ask WHY it is there. The reason determines the fix.

MECHANISM 1

MCAR - Missing Completely at Random

- Missingness is unrelated to any variable, observed or not
- Example: a lab sample is dropped and broken by accident
- The safest case - the remaining data is still representative
- Dropping rows is defensible here (if you can spare them)

MECHANISM 2

MAR - Missing at Random

- Missingness depends on OBSERVED variables
- Example: men are less likely to complete a depression questionnaire
- Missingness is predictable from other columns you do have
- Model-based imputation (K-NN, regression) works well

MECHANISM 3

MNAR - Missing Not at Random

- Missingness depends on the MISSING VALUE ITSELF
- Example: very high earners decline to state their income
- The hardest case - the gap is systematically biased
- Dropping or naive imputation both distort your results
- Often best handled by adding a 'was_missing' indicator column

WHY CLASSIFY

The mechanism decides the treatment

- MCAR → deletion is acceptable
- MAR → impute using the other columns
- MNAR → the missingness is information; flag it explicitly
- You usually cannot prove which one you have - reason about the data-generating process

2.3 MISSING VALUES

UNIT 2

Imputation Strategies and Their Trade-offs

There is no universally best method - there is a best method for your mechanism.

Method	How it works	When to use it	What it costs you
Drop rows	Delete any row containing a missing value	MCAR, and less than ~5% of rows affected	Loses data; badly biased if not MCAR
Mean imputation	Fill with the column mean	Numeric, roughly symmetric, few gaps	Shrinks variance; distorted by outliers
Median imputation	Fill with the column median	Numeric and skewed - the safer default	Shrinks variance; ignores other columns
Mode imputation	Fill with the most frequent value	Categorical features	Over-represents the majority category
K-NN imputation	Average the k most similar rows	MAR, and correlations between features matter	Computationally heavy on large N
Indicator column	Add was_missing = 1/0, then impute	MNAR - when the gap itself is informative	Adds a feature; needs enough data to learn from

2.3 OUTLIERS

UNIT 2

Detecting Outliers - Two Standard Methods

METHOD 1 · Z-SCORE (PARAMETRIC - ASSUMES ROUGHLY NORMAL DATA)

$$Z = (X - \mu) / \sigma \quad \text{flag as outlier if } |Z| > 3$$

$\pm 3\sigma$ covers 99.73% of a Normal distribution, so $|Z| > 3$ is genuinely rare.

METHOD 2 · INTERQUARTILE RANGE (NON-PARAMETRIC - NO DISTRIBUTION ASSUMED)

$$\text{IQR} = Q_3 - Q_1 \quad \text{Lower} = Q_1 - 1.5 \cdot \text{IQR} \quad \text{Upper} = Q_3 + 1.5 \cdot \text{IQR}$$

Anything outside [Lower, Upper] is an outlier. This is what a box plot draws.

CHOOSING BETWEEN THEM

Z-score use when the feature is roughly symmetric and bell-shaped

IQR use when the feature is skewed - it is the safer default

Why IQR is robust quartiles barely move when you add an extreme value; μ and σ are both dragged by it

The irony outliers inflate σ , which raises the $|Z| > 3$ threshold, which can hide the very outliers you are hunting

Q_1 / Q_3 the 25th and 75th percentiles

2.3 OUTLIERS

UNIT 2

Worked Example: Find the Outliers Using the IQR Rule

THE PROBLEM

- A sensor logs these temperature readings (°C):
- 18, 19, 20, 21, 22, 22, 23, 24, 25, 210
- $n = 10$ readings, already sorted
- Task: use the IQR rule to identify any outliers

WHY IT MATTERS

- This is the masking effect in action - a compelling reason to prefer IQR
- Note the fences are computed from the data, not chosen by you
- Physical check: 210 °C from a room sensor is a hardware fault, not weather

STEP-BY-STEP SOLUTION

Step 1 Find Q_1 (25th percentile). With $n = 10$, Q_1 lies between the 2nd and 3rd values: $Q_1 = 19.75$

Step 2 Find Q_3 (75th percentile), between the 8th and 9th values: $Q_3 = 24.25$

Step 3 $IQR = Q_3 - Q_1 = 24.25 - 19.75 = 4.5$

Step 4 Lower fence = $Q_1 - 1.5 \times IQR = 19.75 - 6.75 = 13.0$

Step 5 Upper fence = $Q_3 + 1.5 \times IQR = 24.25 + 6.75 = 31.0$

Step 6 Any reading outside $[13.0, 31.0]$ is an outlier. 210 lies far outside.

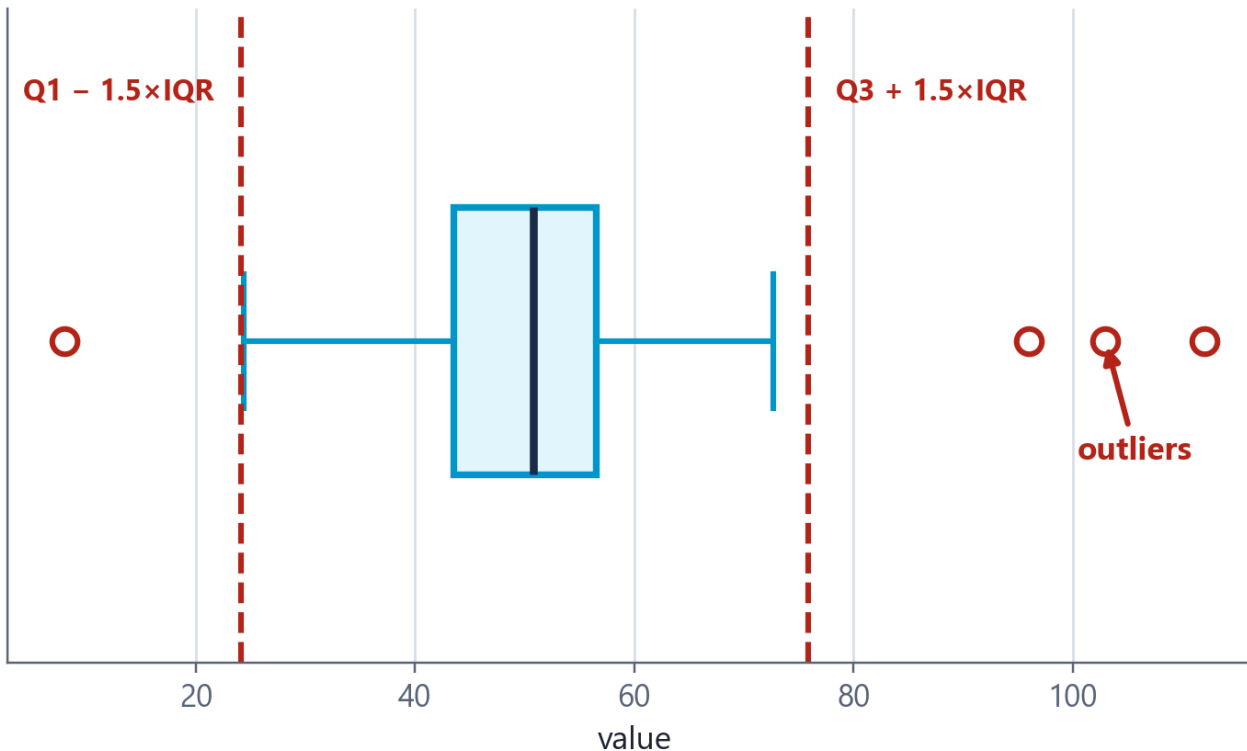
Cross-check By Z-score: $\mu \approx 40.4$, $\sigma \approx 56.9$, so Z for 210 ≈ 2.98 - which does NOT exceed 3. The single outlier inflated σ and hid itself.

ANSWER 210 °C is an outlier by the IQR rule — and the Z-score method misses it

2.3 OUTLIERS

The IQR Rule, Drawn

UNIT 2



HOW TO READ THIS FIGURE

The box spans Q1 to Q3 - the middle 50% of the data.

The line inside the median, not the mean.

The whiskers reach to the last point within $1.5 \times IQR$.

The red dashed lines the fences. $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$.

The circles points beyond a fence. Flagged as outliers - NOT automatically wrong.

TAKEAWAY The IQR rule FLAGS candidates. Deciding what to do with them is a judgement call, not a formula.

2.3 OUTLIERS

UNIT 2

Never Delete an Outlier Reflexively

COMMON PITFALL · Sometimes the outlier is the whole point

The reflex to resist students learn the IQR rule and immediately delete everything outside the fences. That is not outlier treatment - it is data destruction.

When deletion is right the value is physically impossible or a known recording error: a human age of 300, a room temperature of 210 °C, a negative quantity sold.

When deletion is catastrophic fraud detection, network intrusion, disease screening, equipment failure prediction. In all of these, the rare extreme case is exactly what you are trying to find.

Alternative 1 - Winsorising / capping clip values to the fence rather than removing the row. Keeps the record and its other features, tames the extreme.

Alternative 2 - Transform apply a log transform to compress a long right tail. Often turns a skewed feature into a well-behaved one.

Alternative 3 - Use a robust model tree-based models are far less sensitive to outliers than linear regression. Sometimes the fix is the algorithm, not the data.

ASK THE CLASS A bank's transaction data shows a ₹40 lakh transfer among thousands of ₹500 transactions. Outlier or signal? What do you need to know to decide?

ROBOTICS LINK

Robot Sensor Data Is Exactly This Problem - Only Worse

UNIT 2



HOW TO READ THIS DIAGRAM

Dropped packets genuinely MCAR - a lost UDP frame knows nothing about the world.

Spikes sun glare on a camera, a wheel hitting a kerb. Real readings of a real event.

Silent gaps MNAR, and the dangerous one - lidar returns NOTHING off glass and matt-black surfaces.

So the missingness correlates with exactly what you most need to detect.

Never blanket-impute a sensor stream. Ask WHY each gap exists first.

DID YOU KNOW?

Lidar systematically under-returns from black cars and glass. The gap in the data is correlated with the hazard - which is the textbook definition of MNAR.

THINK OF IT LIKE a torch that goes out. and only ever goes out in the darkest room.

TAKEAWAY A lidar that returns nothing where a glass door is has not given you a missing value. It has given you the most important reading of the day.

04

SECTION

Feature Engineering: Scaling, Encoding and Selection

IN THIS SECTION

- Why distance-based algorithms need scaling
- Min-Max normalisation vs Standardisation
- Label/Ordinal encoding vs One-Hot Encoding
- The dummy variable trap
- Filter, wrapper and embedded feature selection
- Data leakage - the most expensive mistake in this unit

2.4 FEATURE SCALING

UNIT 2

Feature Scaling - Two Formulas You Must Know

Salary is in the tens of thousands; Age is in the tens. Without scaling, Salary dominates every distance calculation.

MIN-MAX NORMALISATION - RESCALES TO A FIXED RANGE [0, 1]

$$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Preserves the shape of the distribution. Highly sensitive to outliers, because X_{min} and X_{max} are themselves extreme values.

STANDARDISATION / Z-SCORE SCALING - MEAN 0, STANDARD DEVIATION 1

$$X_{\text{std}} = (X - \mu) / \sigma$$

Not bounded to a fixed range. Far more robust to outliers. The usual default.

WHICH ONE, WHEN

Use Min-Max when you need a bounded range - image pixels, neural network inputs

Use Standardisation as the general default, especially with outliers present

Algorithms that NEED scaling K-NN, K-Means, SVM, PCA, and any gradient-descent model

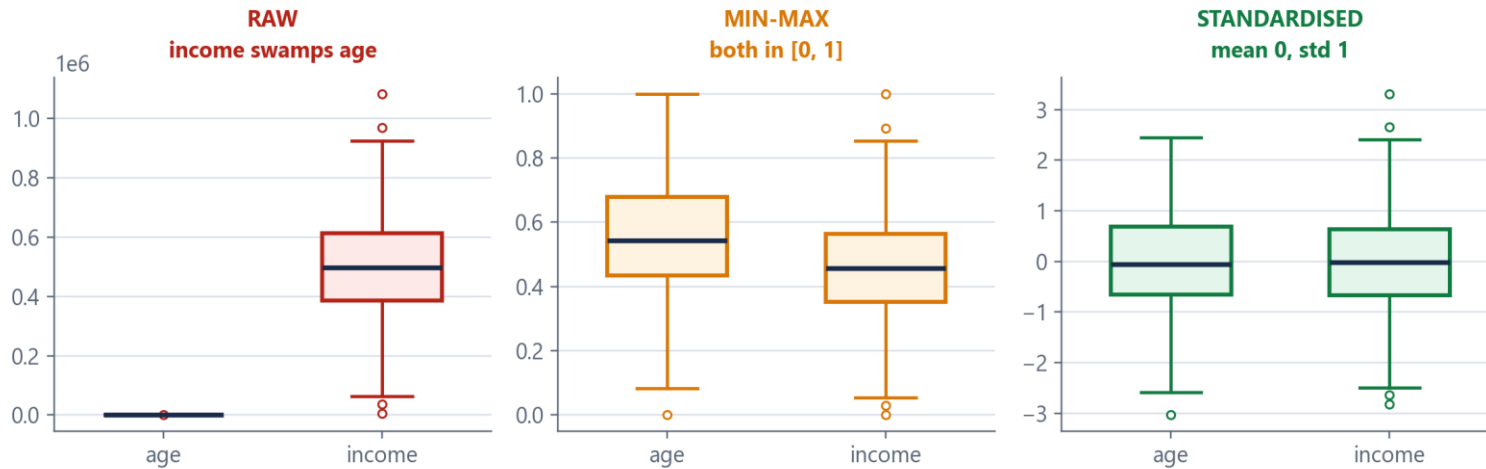
Algorithms that do NOT decision trees, random forests - they split on order, not magnitude

Never scale the target variable in a classification task

2.4 FEATURE SCALING

Raw, Min-Max and Standardised - The Same Two Features

UNIT 2



HOW TO READ THIS FIGURE

Left - RAW age is the flat line at the bottom. It has not vanished; income's scale has crushed it.

Middle - MIN-MAX both squeezed into [0, 1]. Bounded and intuitive.

Right - STANDARDISED mean 0, std 1. Outliers stay visible.

Why it matters any distance-based method - K-NN, K-Means, SVM - is decided by the largest-range feature.

Which to use z-score by default; min-max when you need bounded values.

TAKEAWAY In the left panel age is IN the model and has no influence whatsoever. Nothing warns you.

2.4 FEATURE SCALING

Data Leakage in Scaling - The Mistake Everyone Makes Once

UNIT 2

COMMON PITFALL · Fit the scaler on TRAIN only. Never on the full dataset.

The wrong way `scaler.fit_transform(X)` on the entire dataset, and THEN split into train and test.

Why it is wrong μ and σ were computed using the test set. Information about the test distribution has leaked into your training pipeline.

What you observe excellent validation accuracy, then a disappointing collapse in production. The gap is the leak.

The right way split first. Then `scaler.fit(X_train)` — `transform(X_train)` — `transform(X_test)`. The test set is transformed using the TRAINING statistics only.

The general principle the test set must simulate data you have never seen. Any statistic computed from it - a mean, a median, a category list, a scaler - is a leak.

Applies equally to imputation values, encoders, feature selection, and any other step that learns something from the data.

The habit that prevents it forever

- Use a scikit-learn Pipeline: it enforces fit-on-train automatically
- Inside cross-validation, the Pipeline refits the scaler on each training fold
- If you write manual preprocessing, split on line 1 of your script

2.4 FEATURE SCALING

UNIT 2

Worked Example: Min-Max vs Standardisation on the Same Data

THE PROBLEM

- Ages of five customers: 20, 25, 30, 35, 60
- $n = 5$, $\min = 20$, $\max = 60$
- $\text{mean } \mu = 34$, $\text{standard deviation } \sigma \approx 14.28$
- Task: scale the value $X = 25$ with both methods

WHY IT MATTERS

- One extreme value distorts Min-Max far more than Standardisation
- Min-Max guarantees the $[0,1]$ range; Standardisation does not
- This is why Standardisation is the safer default

STEP-BY-STEP SOLUTION

Min-Max $X_{\text{norm}} = (25 - 20) / (60 - 20) = 5 / 40 = 0.125$

Standardisation $X_{\text{std}} = (25 - 34) / 14.28 = -9 / 14.28 \approx -0.63$

Now scale the whole column, Min-Max $20 \rightarrow 0.000, 25 \rightarrow 0.125, 30 \rightarrow 0.250, 35 \rightarrow 0.375, 60 \rightarrow 1.000$

Notice the crowding the four ordinary ages are squeezed into $[0, 0.375]$ because the single value 60 defines the top of the range.

Standardisation of the column $20 \rightarrow -0.98, 25 \rightarrow -0.63, 30 \rightarrow -0.28, 35 \rightarrow +0.07, 60 \rightarrow +1.82$

Notice the difference values spread around 0, and the extreme age is clearly flagged at $+1.82\sigma$ rather than compressing everything else.

ANSWER Min-Max gives 0.125; Standardisation gives -0.63

Why Scaling Is Not Optional in Sensor Fusion

Lidar range

0 – 100 metres

Camera pixel intensity

0 – 255 unitless

IMU angular rate

–20 – +20 radians / second

Wheel odometry

0 – 3 metres / second

Battery level

0 – 100 percent

five features, five different physical dimensions

HOW TO READ THIS DIAGRAM

Look at the ranges lidar spans 100, odometry spans 3. A 33× difference before you start.

Worse than usual these are not just different SIZES, they are different DIMENSIONS.

Metres, radians per second, percent adding their squares is physically meaningless.

Unscaled any distance-based model is a lidar model with four decorative extra columns.

So standardise $z = (x - \mu)/\sigma$ turns every sensor into 'how unusual is this reading', which IS comparable.

DID YOU KNOW?

Standardisation is not just a scale fix here - it is a UNIT fix. A z-score is dimensionless, which is the only reason summing across sensors means anything.

THINK OF IT LIKE scoring a decathlon. you cannot add metres to seconds - you convert both to points first.

TAKEAWAY Standardising is what makes it legitimate to compare a reading in metres with a reading in radians per second.

CHECK YOUR UNDERSTANDING

Quick Check - Section 3

UNIT 2

QUICK CHECK

A salary column is strongly right-skewed and contains a few extreme values. You need to fill its missing entries. What is the best simple choice, and why?

- A** Mean - it uses all the data, so it is most informative
- B** Median - it is robust to the skew and the extreme values
- C** Zero - it is neutral and adds no assumption
- D** Drop every row with a missing salary

ANSWER: B The median is a robust statistic: extreme values barely move it, whereas the mean is dragged upward by them. Imputing the mean of a right-skewed salary column inserts a value that resembles no actual employee. Zero would be a fabricated data point, and dropping rows loses data and biases the sample unless the data is MCAR.

2.4 FEATURE ENCODING

UNIT 2

Feature Encoding - Turning Categories into Numbers

Machine learning models consume numbers. The question is which numbers preserve the meaning of your categories.

Label / Ordinal Encoding

What it does maps each category to an integer: Low → 0, Medium → 1, High → 2.

Use it when the categories have a genuine ORDER - sizes, ratings, education levels, severity grades.

The danger applied to unordered categories it invents a false ranking. Encoding Red=0, Green=1, Blue=2 tells the model that Blue > Red, and that Green is the average of Red and Blue. Both are nonsense.

Columns added none - one column stays one column.

Scikit-learn LabelEncoder (for targets), OrdinalEncoder (for features).

One-Hot Encoding (OHE)

What it does creates one binary 0/1 column per category. Colour becomes is_Red, is_Green, is_Blue.

Use it when the categories are NOMINAL - unordered. Colour, city, department, product type.

The danger high cardinality. A column with 5,000 distinct postcodes becomes 5,000 columns - the curse of dimensionality.

Columns added K categories become K columns (or K-1, see the trap).

Scikit-learn OneHotEncoder, or pandas get_dummies().

DECIDE For each of these, ordinal or one-hot? T-shirt size · City · Customer satisfaction rating · Blood group · Degree classification

2.4 FEATURE ENCODING

UNIT 2

The Dummy Variable Trap

EXAM TIP · Always drop one column after one-hot encoding

The setup one-hot encode Colour with three categories and you get is_Red, is_Green, is_Blue.

The problem these three columns always sum to exactly 1. Any one of them is perfectly predictable from the other two:
$$\text{is_Blue} = 1 - \text{is_Red} - \text{is_Green}.$$

The technical name perfect multicollinearity. The design matrix becomes singular, so $(X^T X)$ cannot be inverted and linear regression coefficients become unstable or undefined.

The fix keep only $K-1$ columns. In scikit-learn use `OneHotEncoder(drop='first')`; in pandas use `get_dummies(drop_first=True)`.

No information is lost if `is_Red = 0` and `is_Green = 0`, the model already knows the colour is Blue. The dropped category becomes the reference level.

When you can ignore it tree-based models and regularised regression handle it without breaking - but dropping is still tidier and cheaper.

2.4 FEATURE SELECTION

UNIT 2

Feature Selection - Three Families of Methods

More features is not better. Irrelevant features add noise, cost and overfitting.

FAMILY 1

Filter methods

- Rank features by a statistical score, independently of any model
- Pearson correlation with the target
- ANOVA F-test (numeric feature, categorical target)
- Chi-square χ^2 (categorical feature, categorical target)
- FAST and model-agnostic; ignores feature interactions

FAMILY 2

Wrapper methods

- Search over feature SUBSETS, scoring each with an actual model
- Recursive Feature Elimination (RFE): fit, drop the weakest, repeat
- Forward selection and backward elimination
- Finds interactions filters miss
- EXPENSIVE - trains many models; risks overfitting the selection

FAMILY 3

Embedded methods

- Selection happens automatically during training
- L1 / Lasso penalty shrinks useless coefficients to exactly zero
- Tree-based Gini or permutation importance
- Best balance of cost and quality - usually the practical choice
- You get selection for free with the model fit

WHY SELECT AT ALL

The benefits

- Reduces overfitting - fewer parameters to memorise noise
- Faster training and faster inference
- Cheaper data collection in production
- More interpretable models
- Mitigates the curse of dimensionality

CHECK YOUR UNDERSTANDING

Quick Check - Section 4

UNIT 2

QUICK CHECK

You one-hot encode a 'City' column with 4 categories and feed all 4 new columns into a linear regression. What problem have you created?

- A** The curse of dimensionality - too many features
- B** The dummy variable trap - perfect multicollinearity
- C** Data leakage from the test set into training
- D** Nothing; one-hot encoding always requires all K columns

ANSWER: B The four indicator columns always sum to 1, so any one is perfectly predictable from the other three. This is perfect multicollinearity - $X^T X$ becomes singular and the regression coefficients are unstable. Fix it by keeping K-1 columns: drop='first'.

05

SECTION

Training and Validation

IN THIS SECTION

- Why you must hold out data you never touch
- The train-test split, and typical proportions
- K-Fold Cross Validation and why it is more reliable
- Stratified K-Fold for imbalanced classes
- Putting it together: the scikit-learn Pipeline

2.5 TRAINING AND VALIDATION

UNIT 2

Train-Test Split - The Basic Protocol

How it works

The split partition the data into a training set (typically 70-80%) and a held-out test set (typically 20-30%).

Train the model sees only the training set and learns its parameters from it.

Test the model is evaluated ONCE on the test set, which simulates unseen future data.

random_state fix the seed so the split is reproducible - your results must be repeatable.

stratify=y for classification, preserves the class proportions in both sets.

What it protects you from

The core problem a model evaluated on its own training data is marking its own homework. It can score perfectly by memorising.

Overfitting made visible high training accuracy with low test accuracy is the diagnostic signature. You cannot see it without a held-out set.

The cardinal rule look at the test set ONCE, at the very end. Tuning against it repeatedly turns it into a second training set.

Validation set if you need to tune, carve a third VALIDATION set out of training - or use cross-validation.

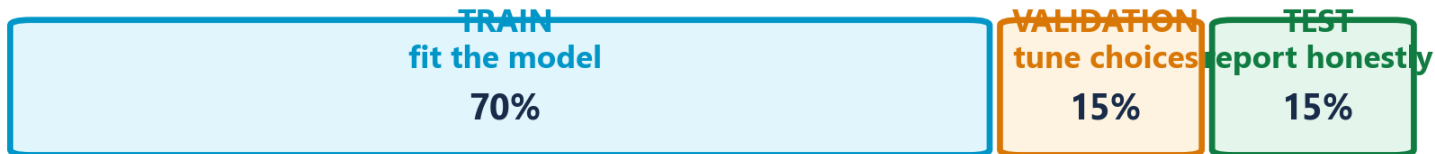
Time-series exception never split randomly. Split chronologically, or you train on the future.

ASK THE CLASS Your model scores 99% on training data and 62% on test data. What is happening, and what would you do first?

2.5 TRAINING AND VALIDATION

UNIT 2

The Three-Way Split, and the Rule About the Test Set



the test set is opened ONCE, at the very end, and never again

HOW TO READ THIS FIGURE

Train the model fits its parameters here.

Validation you compare models and tune hyperparameters here. You may use it many times.

Test touched ONCE, at the end, to report an honest number.

Why three and not two every time you look at a set and change something, you leak information into your choices.

The discipline if you tune against the test set, your reported accuracy is fiction.

TAKEAWAY Train fits. Validation chooses. Test reports - once.

2.5 TRAINING AND VALIDATION

K-Fold Cross Validation

UNIT 2

A single train-test split gives one number, and that number depends on luck. K-fold averages away the luck.

THE CROSS-VALIDATED PERFORMANCE ESTIMATE

$$E_{CV} = (1/K) \cdot \sum E_i \quad \text{for } i = 1 \dots K$$

Split into K equal folds. Train on K-1, validate on the held-out fold. Rotate K times, then average.

WHY IT IS BETTER

Uses all the data every observation serves as validation exactly once

Lower variance averaging K estimates is more stable than one lucky split

Typical K 5 or 10 - a good balance of reliability and compute cost

Cost you train the model K times instead of once

K = n Leave-One-Out CV; nearly unbiased but very expensive

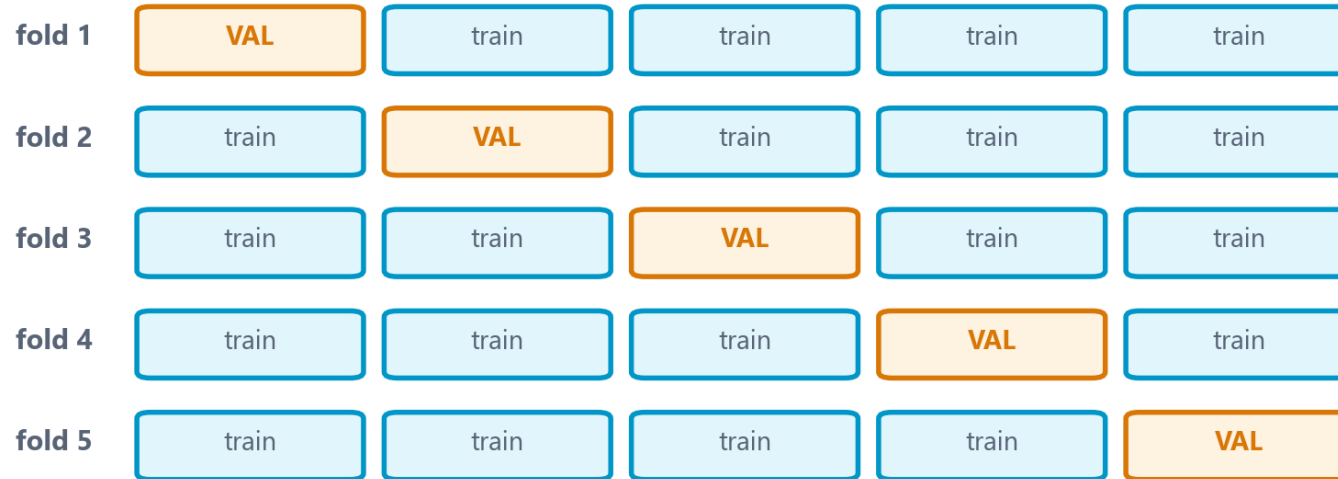
Still needed keep a final untouched test set for the last honest evaluation

2.5 TRAINING AND VALIDATION

UNIT 2

K-Fold Cross Validation, One Row Per Fold

Every row trains on $k-1$ folds and validates on the one held out



Final score = the AVERAGE of the 5 validation scores

HOW TO READ THIS FIGURE

Five rows the same data, split five ways. Each row is one complete experiment.

Gold block the fold held out for validation in that run.

Blue blocks the $k-1$ folds used for training in that run.

Every point is validated exactly once and trained on $k-1$ times.

The result average the k scores. You also get a standard deviation - the stability of the model.

TAKEAWAY A single train/test split is one opinion. K-fold gives you five, and tells you how much they disagree.

2.5 TRAINING AND VALIDATION

Stratified K-Fold - Essential for Imbalanced Data

UNIT 2

EXAM TIP · When classes are rare, random folds are not good enough

The problem in fraud detection roughly 0.1% of transactions are fraudulent. Split randomly into 10 folds and some folds may contain ZERO fraud cases.

Why that breaks things a validation fold with no positive examples cannot measure recall at all. Your metric is undefined or meaningless.

The fix Stratified K-Fold preserves the class proportions in every fold. If the full dataset is 0.1% fraud, every fold is 0.1% fraud.

Use it for any classification task, and always when classes are imbalanced. There is essentially no downside.

In scikit-learn StratifiedKFold, or pass stratify=y to train_test_split.

The same idea applies to the initial train-test split, not only to cross-validation folds.

CONNECT IT Unit 5 covers precision and recall. Why does a fold with zero positive cases make recall impossible to compute?

2.5 TRAINING AND VALIDATION

Activity: Diagnose the Broken Pipeline

UNIT 2

PAIR WORK · 10 minutes

Here is a preprocessing workflow written by a previous student. It reports 96% accuracy in development and 71% in production. Find every mistake.

1

The code they wrote

- 1. Load the full dataset
- 2. Fill all missing values with the column mean
- 3. Scale every feature with MinMaxScaler
- 4. One-hot encode City into all 8 columns
- 5. Split into train and test
- 6. Train a K-NN classifier

2

Find the leaks

- Which steps used information from the test set?
- At which line number should the split have happened?
- What is the consequence of each leak?

3

Find the other bugs

- Is mean the right imputation choice here?
- Is MinMax the right scaler if outliers exist?
- What is wrong with all 8 one-hot columns?

4

Rewrite it

- Put the steps in the correct order
- Name the scikit-learn class for each step
- State which class prevents the leak structurally

DELIVERABLE Each pair reports ONE mistake and its fix. No repeats - so listen to the pairs before you.

CHECK YOUR UNDERSTANDING

Quick Check - Section 5

UNIT 2

QUICK CHECK

You scale your features on the full dataset, then split into train and test, then report 97% test accuracy. Why is that number not trustworthy?

- A** Because 97% is too high to be realistic for any model
- B** Because the scaler learned μ and σ from the test set - the test score is leaked
- C** Because you should have used Min-Max scaling instead
- D** Because the test set was too small to give a reliable estimate

ANSWER: B Fitting the scaler before splitting means μ and σ were computed using test data, so information about the test distribution entered the training pipeline. The reported score is optimistically biased. Split first, fit the scaler on train, then transform both - or use a Pipeline, which enforces this automatically.

COMMON MISTAKES

Common Mistakes to Avoid

UNIT 2

Every one of these is a real error students make in this unit - and every one is avoidable.

MISTAKE 1**Fitting the scaler before splitting**

- The scaler sees the test set - that is leakage
- Use a Pipeline and it cannot happen

MISTAKE 2**Filling missing values by reflex**

- Ask WHY each gap exists first
- MNAR: the absence is itself the signal

MISTAKE 3**Deleting outliers to tidy the data**

- In fraud and fault detection they ARE the answer
- Flagged is not the same as condemned

MISTAKE 4**Trusting a correlation matrix**

- It ignores every categorical column
- And every non-linear relationship. Plot it.

THE FOUR THAT RUIN PROJECTS

X scale before split → leakage

X fillna without asking why

X delete the outliers

X trust r without plotting

RESEARCH LENS

Research Lens

UNIT 2

Where this topic is heading - so you know what you are being prepared for.

DIRECTION 1**Data-centric AI**

- Improve the DATA, hold the model fixed
- Often beats months of model tuning
- A deliberate reversal of the usual emphasis

DIRECTION 2**Automated data quality**

- Tools that detect drift and schema breaks in production
- Monitoring as a first-class part of the pipeline
- Exactly what Google Flu Trends lacked

DIRECTION 3**Synthetic and privacy-safe data**

- Generating data when real data cannot be shared
- Important in health and finance
- Open question: does it carry the same biases?

WHERE THIS IS GOING

→ fix the data, not the model

→ monitor drift automatically

? does synthetic data inherit bias?

UNIT II RECAP

Unit II - The Six Ideas Worth Remembering

UNIT 2

1**Data preparation is the majority of the work**

70-80% of real project time. NumPy computes, Pandas organises, Matplotlib reveals - and vectorisation is why NumPy is ~100x faster than a Python loop.

2**EDA before modelling, always**

Univariate, bivariate, multivariate. And never trust a correlation coefficient without plotting the scatter - $r = 0$ rules out only a LINEAR relationship.

3**Ask why a value is missing before you fill it**

MCAR allows deletion, MAR allows model-based imputation, MNAR means the missingness itself is information. Median beats mean when the data is skewed.

4**Outliers need judgement, not a rule**

IQR fences are $Q_1 - 1.5 \cdot \text{IQR}$ and $Q_3 + 1.5 \cdot \text{IQR}$, and IQR is more robust than Z-score. But in fraud and fault detection, the outlier IS the signal.

5**Scale for distance-based models; encode by whether order is real**

Standardisation is the safe default. Ordinal encoding for ordered categories, one-hot for nominal - and drop one column to avoid the dummy variable trap.

6**Split first, and let a Pipeline enforce it**

Every statistic learned from data - mean, σ , median, category list - must come from training data only. Use stratified K-fold, and touch the test set once.

NEXT UP Unit III - Supervised Machine Learning: linear, polynomial and decision-tree regression; logistic regression, K-NN, decision trees, random forests and Naive Bayes; and the bias-variance tradeoff.